

MODULE 1

DAY 1 • MODULE 1 OF 8

From Vibe Coding to Spec-Driven Development

Why unstructured AI prompting breaks down at scale, and the lifecycle this course spends three days on to fix it.

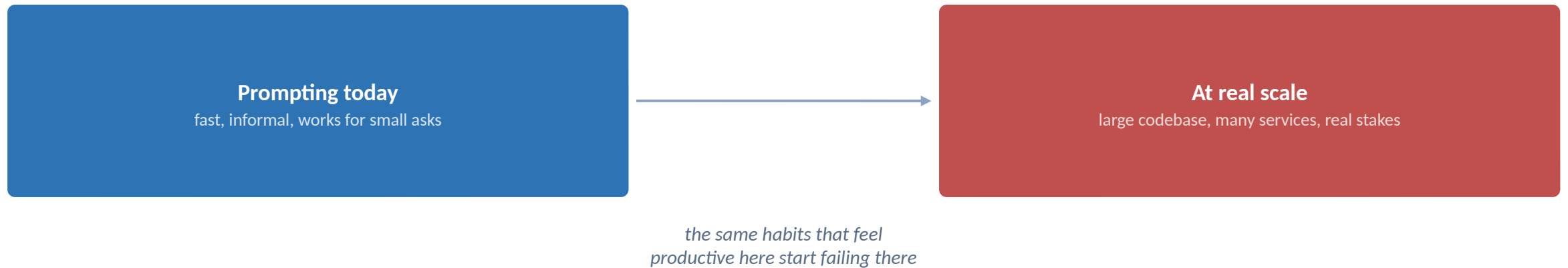
What You'll Learn

- 1 Recognize the failure modes of unstructured AI prompting
- 2 Explain the core thesis of spec-driven development
- 3 Place GitHub Spec Kit and OpenSpec within the wider SDD landscape
- 4 Walk the full SDD lifecycle, stage by stage
- 5 See how this lifecycle maps onto the rest of this course

MODULE 1

WHY THIS MATTERS

You're already prompting AI every day, here's exactly where that breaks.



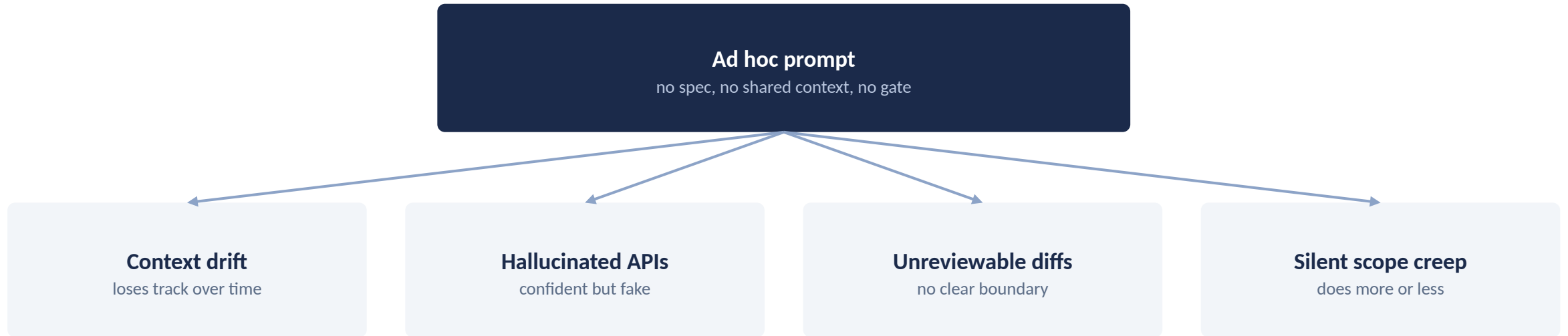
This module names exactly why, then hands you the fix the rest of this course builds on

PART 1

The Problem

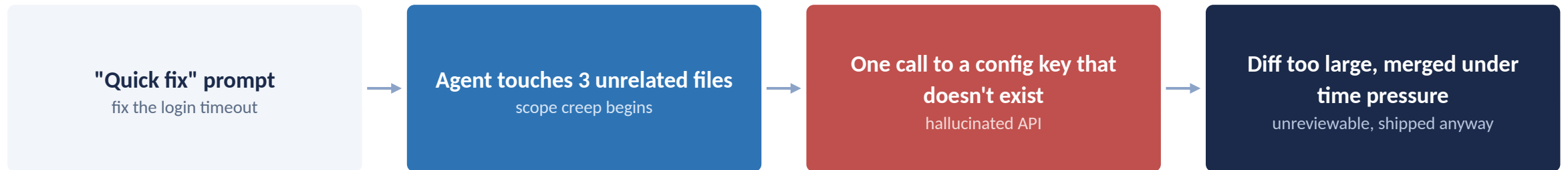
Why unstructured AI prompting breaks down once the stakes get real

Ad Hoc Prompting Fails in Four Predictable Ways



Predictable failures are also preventable ones, that's the whole thesis of this module

These Four Failures Rarely Show Up Alone



Each failure mode makes the next one more likely, that's why they compound instead of staying isolated

CHECKPOINT

Let's Pause and Verify

1 Name one failure mode of ad hoc prompting, and one thing that causes it.

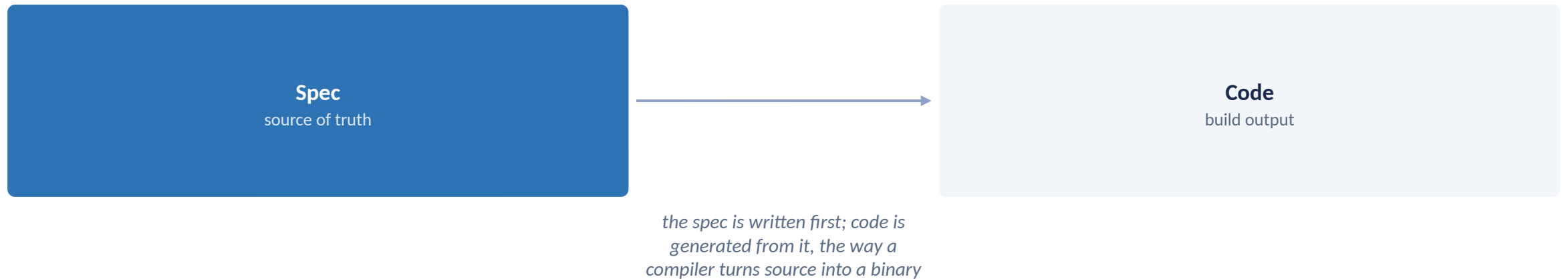
2 Why does "confident but fake" describe a hallucinated API well?

PART 2

The Fix

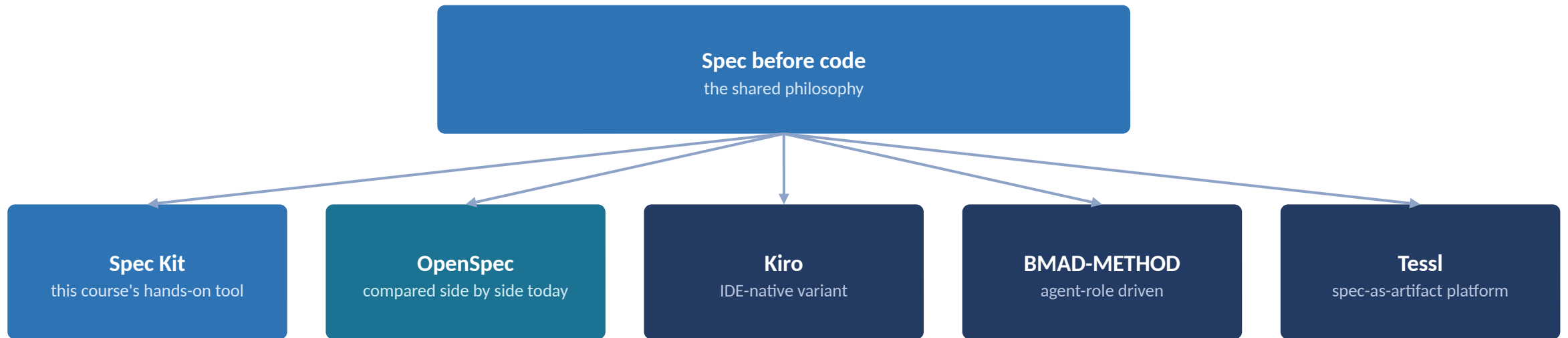
Spec-driven development, and where it sits among other frameworks

The Spec Is the Source of Truth, Code Is the Build Output



Every failure mode from Part 1 traces back to skipping or shortcutting this one move

Multiple Frameworks Implement the Same Core Thesis, Differently



Same philosophy, five different implementations, this course teaches the philosophy through two of them

Vendor-neutral and CLI-first, with OpenSpec alongside for comparison.



Later today's hands-on: scaffold both, and compare how each structures the same workflow

CHECKPOINT

Let's Pause and Verify

1 In one sentence, what's the core thesis of spec-driven development?

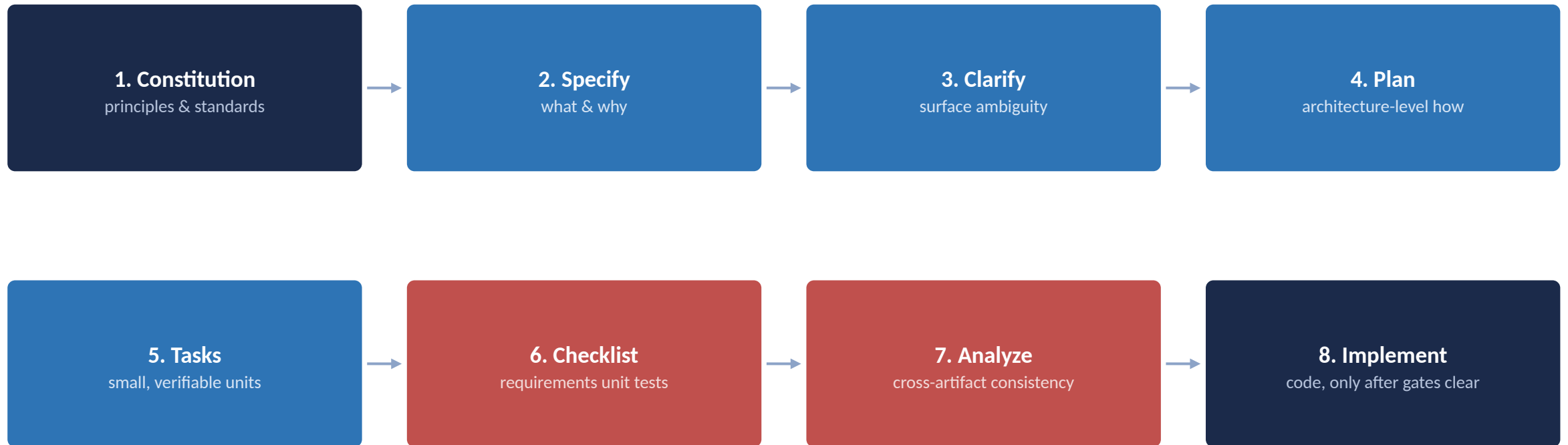
2 Name one other SDD framework besides Spec Kit and OpenSpec.

PART 3

The Map

The full SDD lifecycle, and how the rest of this course is organized around it

Eight Stages, Each Catching a Different Class of Failure



This is the map for the rest of the course, every later module is a deep dive on one part of this sequence

Every Failure Mode From Part 1 Has a Stage That Catches It

Context drift → caught by Specify & Clarify

a written, clarified spec anchors intent so it can't quietly drift turn to turn

Hallucinated APIs → caught by Checklist & Analyze

cross-checking against the spec and constitution surfaces invented interfaces before code exists

Unreviewable diffs → caught by Plan & Tasks

small, independently verifiable units keep every diff small enough to actually review

Silent scope creep → caught by Constitution & Analyze

the constitution sets the boundary of "good", analyze checks the work stayed inside it

None of this is coincidence, the lifecycle was designed backward from these exact four failure modes

This Lifecycle Is the Spine of the Next Three Days

Module 2 • Day 1

Discovery: gather context before writing a single spec

Module 3 • Day 1

Constitution, Specify, Clarify

Modules 4-5 • Day 2

Plan, Tasks, Checklist, Analyze

Module 6 • Day 2

Human review of Implement output

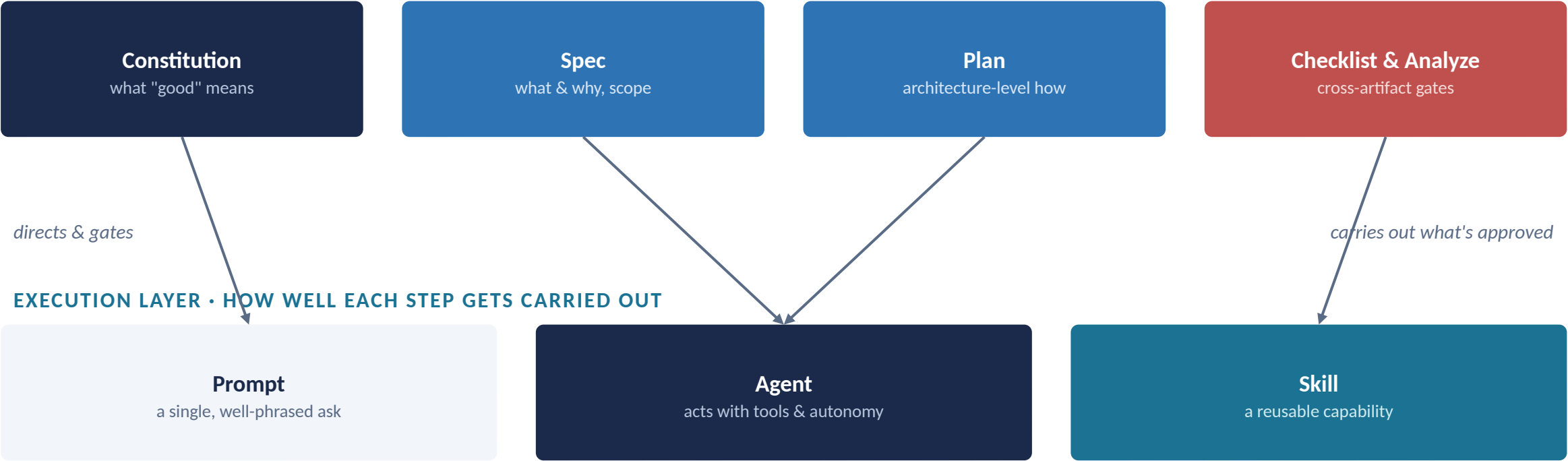
Modules 7-8 & Capstone • Day 3

Multi-agent workflows, pitfalls, full lifecycle applied

Every module from here forward is a deep dive on one or two boxes from the previous slide

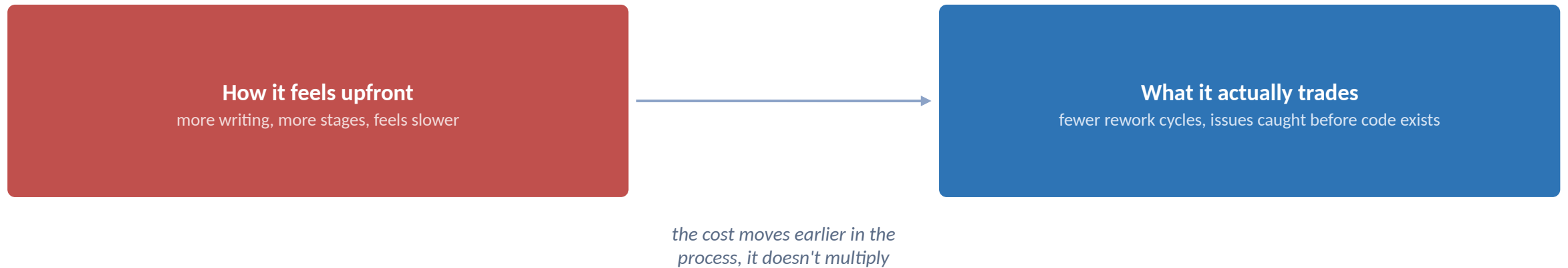
SDD Directs the Execution Layer, It Doesn't Replace It

SDD LAYER • WHAT GETS BUILT, WHY, AND WHETHER IT'S RIGHT



A great agent with a bad spec efficiently builds the wrong thing, you need both layers

SDD front-loads clarity, it doesn't add bureaucracy.



We'll dig into the real token and cost tradeoffs, with case studies, in Module 8

CHECKPOINT

Let's Pause and Verify

- 1 Which lifecycle stage would you expect to catch a hallucinated API, and why?
- 2 In the two-layer picture, which layer decides whether the work should exist at all?

Key Takeaways

- 1 Ad hoc prompting fails in four predictable, and therefore preventable, ways: context drift, hallucinated APIs, unreviewable diffs, silent scope creep.
- 2 SDD's core move: make the spec the source of truth, code the build output, not an afterthought written up after the fact.
- 3 Multiple frameworks share this thesis; this course uses Spec Kit hands-on, with OpenSpec alongside for comparison.
- 4 The eight-stage lifecycle, constitution through implement, is the map for this entire course, and it directs the execution layer (prompts, agents, skills) without replacing it.
- 5 Next: Module 2, Discovery at Scale, how to build real understanding of a large codebase before writing a single spec against it.